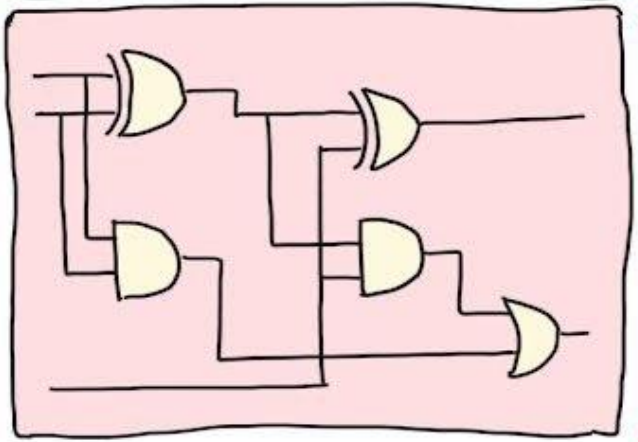
```
int sum (int a, int b) {
    int c = a + b;
    return c;
}
```

Software

```
ADD R0, R0, R1
```

The HW/SW contract



Hardware

Always has been.
JS
php
Wait, its all just assembly?
C
C++
Java
Me
My Professor

Source Code to Execution

Source code fib.c

```
int64_t fib(int64_t n) {
  if (n < 2) return n;
  return (fib(n-1) + fib(n-2));
}
```

```
$ clang fib.c -o fib
```

Compilation

four stages:
- Preprocessing
- Compiling
- Assembling
- Linking

Machine code fib

```
01010101 01001000 10001001
11100101 01010011 01001000
10000011 11101100 00001000
10001001 01111101 11110100
10000011 01111101 11110100
00000001 01111111 00001000
10001011 01000101 11110100
10001001 01000101 11110000
11101011 00011101 10001011
01000101 11110100 10001101
01111000 11111111 11101000
11011011 11111111 11111111
11111111 10001001 11000011
10001011 01000101 11110100
10001101 01111000 11111110
11101000 11001110 11111111
11111111 11111111 00000001
11000011 10001001 01011101
11110000 10001011 01000101
11110000 01001000 10000011
11000100 00001000 01011011
11001001 11000011
```

Hardware interpretation

```
$ ./fib
```

Execution

© 2008–2018 by the MIT 6.172 Lecturers

Assembly Code to Executable

Assembly code fib.s

```
        .globl      _fib
        .p2align    4, 0x90
_fib:   ## @fib
        pushq       %rbp
        movq        %rsp, %rbp
        pushq       %r14
        pushq       %rbx
        movq        %rdi, %rbx
        cmpq        $2, %rbx
        jge         LBB0_1
        movq        %rbx, %rax
        jmp         LBB0_3
LBB0_1:
        leaq        -1(%rbx), %rdi
        callq       _fib
        movq        %rax, %r14
        addq        $-2, %rbx
        movq        %rbx, %rdi
        callq       _fib
        addq        %r14, %rax
LBB0_3:
        popq        %rbx
        popq        %r14
        popq        %rbp
        retq
```

Assembling

```
$ clang fib.s -o fib.o
```

Machine code

```
01010101 01001000
10001001 11100101
01010011 01001000
10000011 11101100
00001000 10001001
01111101 11110100
10000011 01111101
11110100 00000001
01111111 00001000
10001011 01000101
11110100 10001001
01000101 11110000
11101011 00011101
10001011 01000101
11110100 10001101
01111000 11111111
11101000 11011011
11111111 11111111
11111111 10001001
11000011 10001011
01000101 11110100
```

You can edit fib.s and assemble with clang.

© 2008–2018 by the MIT 6.172 Lecturers

RAM as an array of bytes

Content:	FF	00	57	92	B3	8A	... ...	10	46	DC
Address:	000 000 000	000 000 001	000 000 002	000 000 003	000 000 004	000 000 005	... ...	134 217 725	134 217 726	134 217 727

Little-endian encoding

Abstract number: 0xDEADBEEF

Memory representation:

EF	BE	AD	DE
10	11	12	13

Byte address:

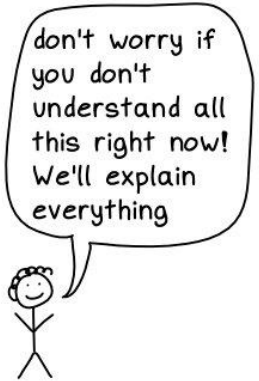
JULIA EVANS
@b0rk

8 bytes, many meanings

The same bytes can mean many different things. Here are 8 bytes and a bunch of things they could potentially mean:

8 bytes → 63 6f 6d 70 75 74 65 72

c	o	m	p	u	t	e	r	8 ASCII characters
99	111	109	112	117	116	101	114	8 8-bit integers
28515		28781		29813		29285		4 unsigned 16-bit integers
1886220131				1919251573				2 unsigned 32-bit integers
8243122740717776739								1 unsigned 64-bit integer
0x72657475706d6f63								a 64-bit pointer
99.111.109.112				117.116.101.114				2 IPv4 addresses
arpl WORD PTR [edi+0x6d],bp			jo 0x7a		je 0x6c		.byte 0x72	x86 machine code
c	o	m	p	u	t	29285		6 ASCII characters + 1 16-bit integer
2.93930e+29				4.54482e+30				2 32-bit floating point numbers
1.144493e+243								1 64-bit floating point numbers
rgba(99, 111, 109, 0.44)				rgba(117, 116, 101, 0.45)				2 RGBA colours



JULIA EVANS
@b0rk

signed vs unsigned integers

there are 2 ways to interpret every integer

unsigned:
- always 0 or more
- example: 8 bit unsigned ints are 0 to 255

signed:
- half positive, half negative
- example: 8 bit signed ints are -128 to 127

negative integers are represented in a counterintuitive way

You might think that this is -5:

10000101

sign bit — 101 in binary is 5

But actually this is -5:

11111011

this looks weird, but we'll explain why!

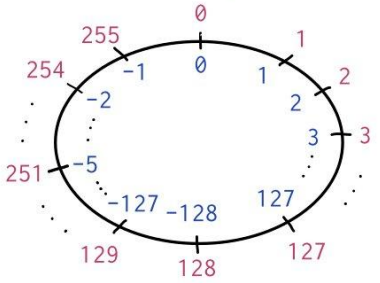
integer addition* wraps around

for example, for 8-bit integers 255 + 1 = 0

for 16-bit integers, 65535 + 1 = 0

*by "addition", we mean "what the x86 add instruction does"

but if 255 + 1 = 0, you could also say 255 = -1



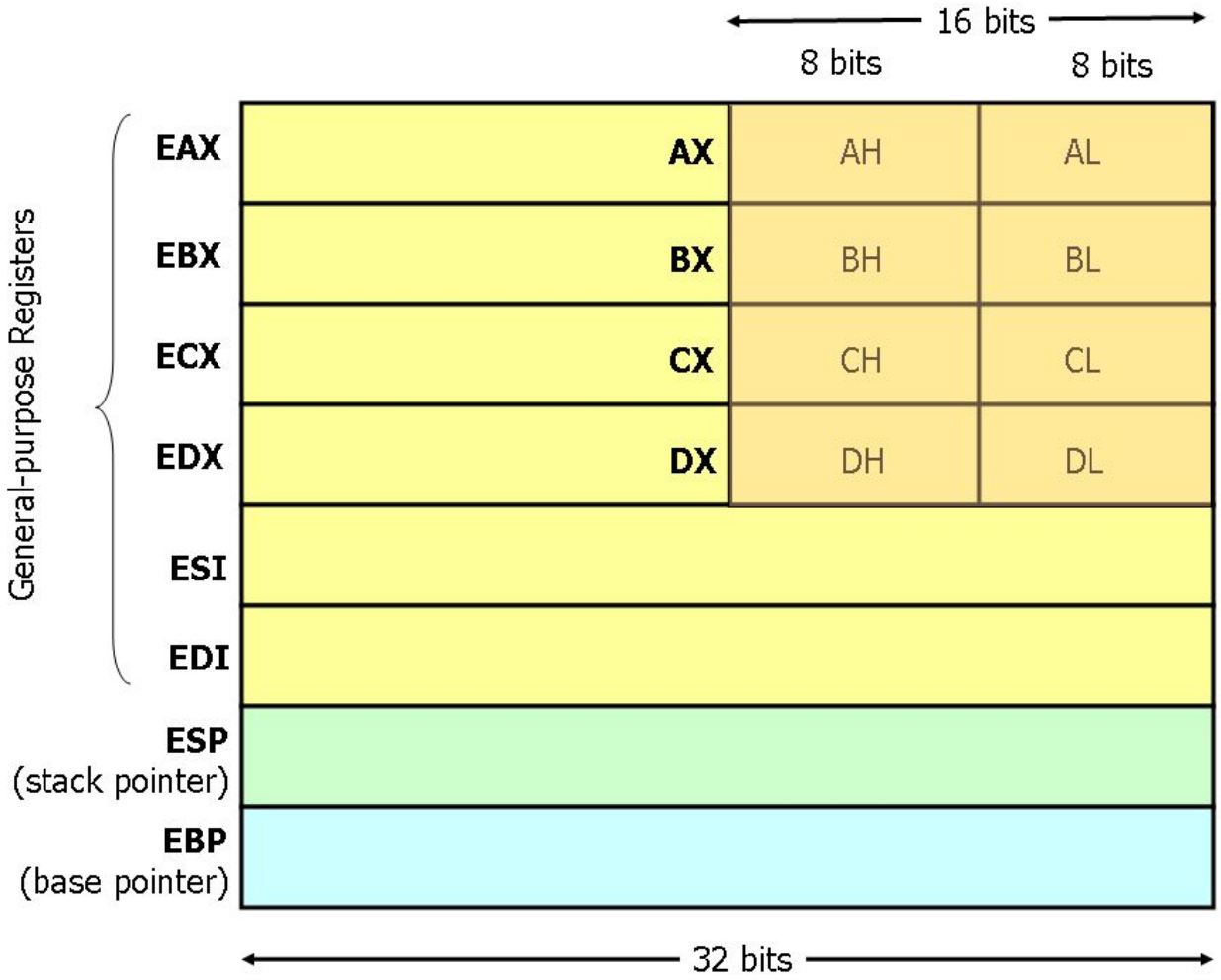
examples of bytes and their signed/unsigned ints

byte	unsigned	signed
00000000	0	0
01111111	127	127
10000000	128	-128
10000001	129	-127
11111011	251	-5
11111111	255	-1

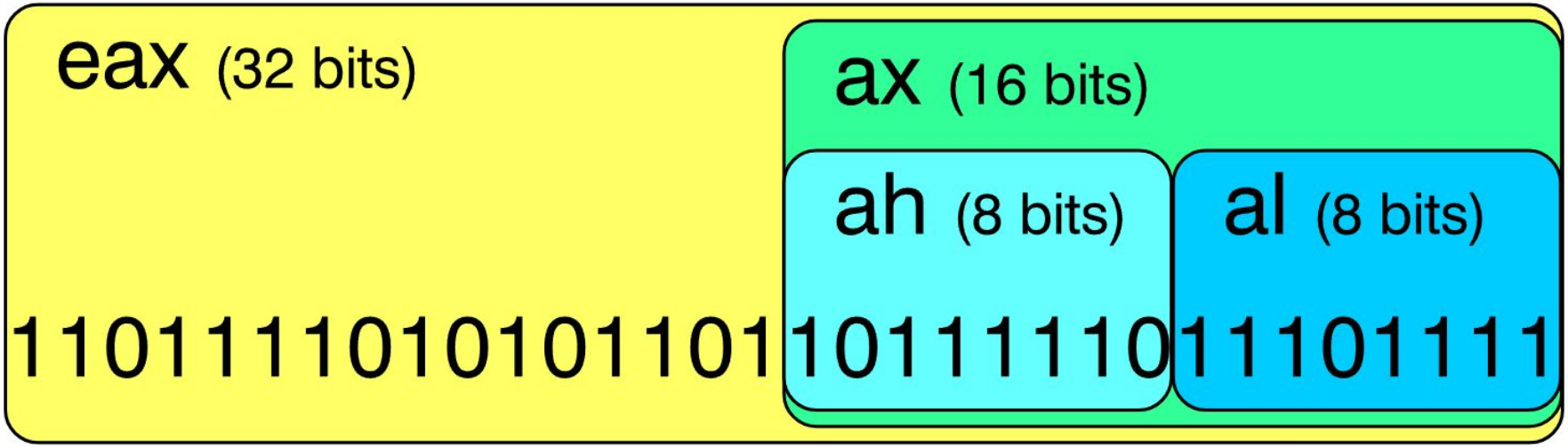
subtract 256 from unsigned numbers to get the signed numbers

this way of handling signed integers is called "two's complement"

It's popular because you can use the same circuits to add signed and unsigned integers. 5 + 255 has exactly the same result as 5 + (-1): they're both 4!


16 bits
8 bits
8 bits
General-purpose Registers
EAX
AX
AH
AL
EBX
BX
BH
BL
ECX
CX
CH
CL
EDX
DX
DH
DL
ESI
EDI
ESP
(stack pointer)
EBP
(base pointer)
32 bits

Register aliasing / sub-registers



AT&T versus Intel Syntax

What does "<op> A, B" mean?

AT&T Syntax B ← B <op> A	Intel Syntax A ← A <op> B
`movl $1, %eax`	`mov eax, 1`
`addl (%ebx,%ecx,0x2), %eax`	`add eax, [ebx+ecx*2h]`
`subq 0x20(%rbx), %rax`	`sub rax, [rbx+20h]`

Generated or used by `clang`, `objdump`, `perf`, 6.172 lectures.

Used by Intel documentation.

© 2008–2018 by the MIT 6.172 Lecturers

Common x86-64 Opcodes

Type of operation		Examples
Data movement	Move	mov
	Conditional move	cmov
	Sign or zero extension	movs, movz
	Stack	push, pop
Arithmetic and logic	Integer arithmetic	add, sub, mul, imul, div, idiv, lea, sal, sar, shl, shr, rol, ror, inc, dec, neg
	Binary logic	and, or, xor, not
	Boolean logic	test, cmp
Control transfer	Unconditional jump	jmp
	Conditional jumps	j<condition>
	Subroutines	call, ret

Note: The subtraction operation "`subq %rax, %rbx`" computes `%rbx = %rbx - %rax`.

© 2008–2018 by the MIT 6.172 Lecturers

x86-64 Data Types

C declaration	C constant	x86-64 size (bytes)	Assembly suffix	x86-64 data type
char	'c'	1	b	Byte
short	172	2	w	Word
int	172	4	l or d	Double word
unsigned int	172U	4	l or d	Double word
long	172L	8	q	Quad word
unsigned long	172UL	8	q	Quad word
char *	"6.172"	8	q	Quad word
float	6.172F	4	s	Single precision
double	6.172	8	d	Double precision
long double	6.172L	16(10)	t	Extended precision

© 2008–2018 by the MIT 6.172 Lecturers

x86-64 Direct Addressing Modes

The operands of an instruction specify values using a variety of addressing modes.

- At most one operand may specify a memory address.

Direct addressing modes

- Immediate: Use the specified value.
- Register: Use the value in the specified register.
- Direct memory: Use the value at the specified memory address.

```
mov eax,$10

mov eax,ebx

mov eax,[$100]
```

© 2008–2018 by the MIT 6.172 Lecturers

x86-64 Indirect Addressing Modes

The x86-64 ISA also supports indirect addressing: specifying a memory address by some computation.

- Register indirect: The address is stored in the specified register.
- Register indexed: The address is a constant offset of the value in the specified register.

```
mov eax,[ebx]

mov eax,[ecx+$100]
```

© 2008-2018 by the MIT 6.172 Lecturers

Base Indexed Scale Displacement

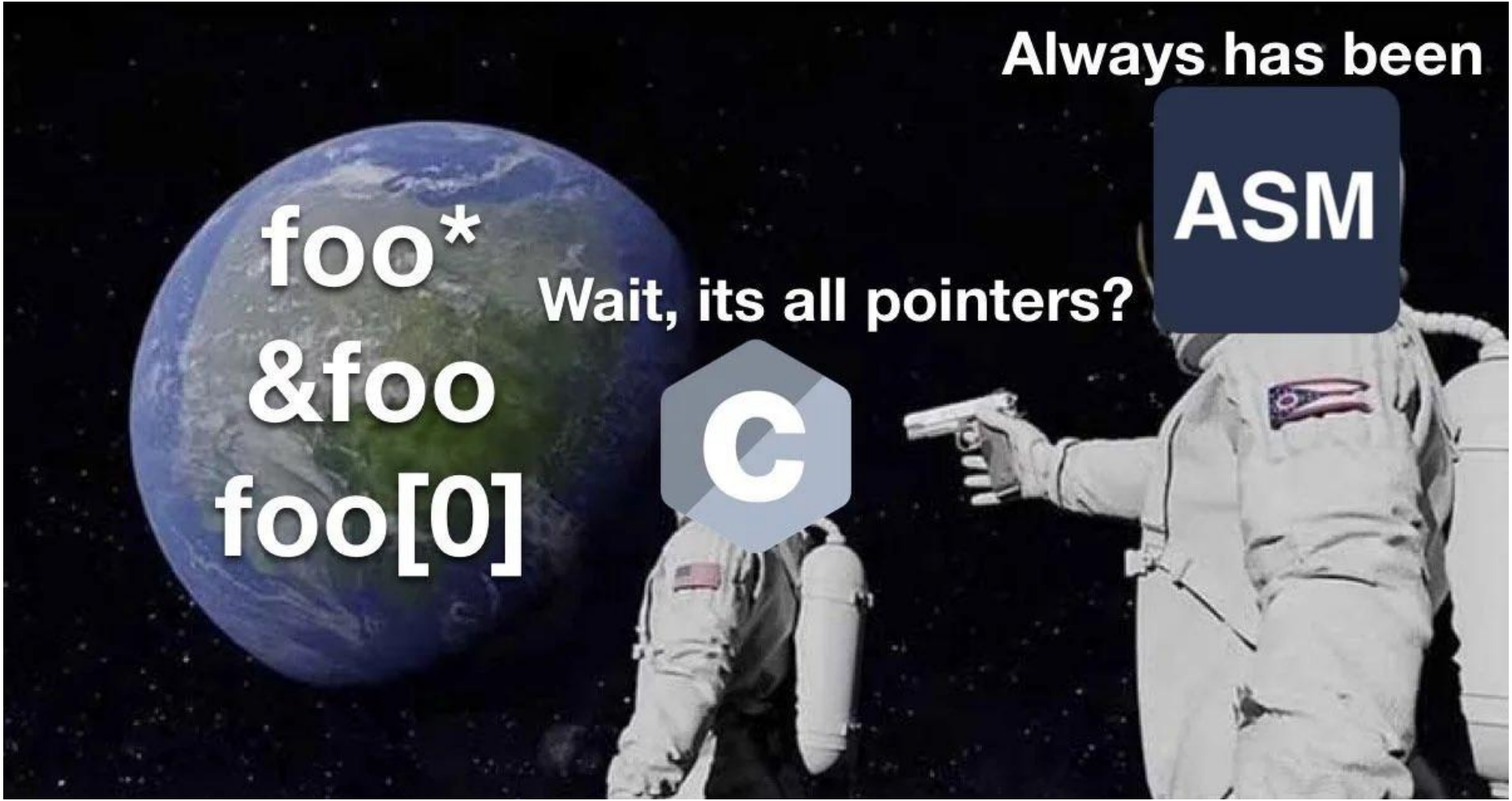
The most general form of indirect addressing supported by x86-64 is the base indexed scale displacement mode.

```
mov eax, [$100+ebx*4+eax]
```

This mode refers to the address Base + Index*Scale + Displacement.

If unspecified, Index and Displacement default to 0, and Scale defaults to 1.

© 2008–2018 by the MIT 6.172 Lecturers


Always has been
ASM
foo*
&foo
foo[0]
Wait, its all pointers?
C

RFLAGS Register

Bit(s)	Abbreviation	Description
0	CF	Carry
1		*Reserved*
2	PF	Parity
3		*Reserved*
4	AF	Adjust
5		*Reserved*
6	ZF	Zero
7	SF	Sign
8	TF	Trap
9	IF	Interrupt enable
10	DF	Direction
11	OF	Overflow
12–63		*System flags or reserved*

The last ALU operation generated a carry or borrow out of the most-significant bit.

The result of the last ALU operation was 0.

The last ALU operation produced a value whose sign bit was set.

The last ALU operation resulted in arithmetic overflow.

© 2008–2018 by the MIT 6.172 Lecturers

Jump Instructions

The x86-64 jump instructions, jmp and j⟨condition⟩, take a label as their operand, which identifies a location in the code.

Example from fib.s

```
 jge  LBB0_1
  …
LBB0_1:
  leaq -1(%rbx), %rdi
```

Example from objdump fib

```
 jge 5 <_fib+0x15>
  …
15:
  leaq -1(%rbx), %rdi
```

- Labels can be symbols, exact addresses, or relative addresses.
- An indirect jump takes as its operand an indirect address. as its operand.

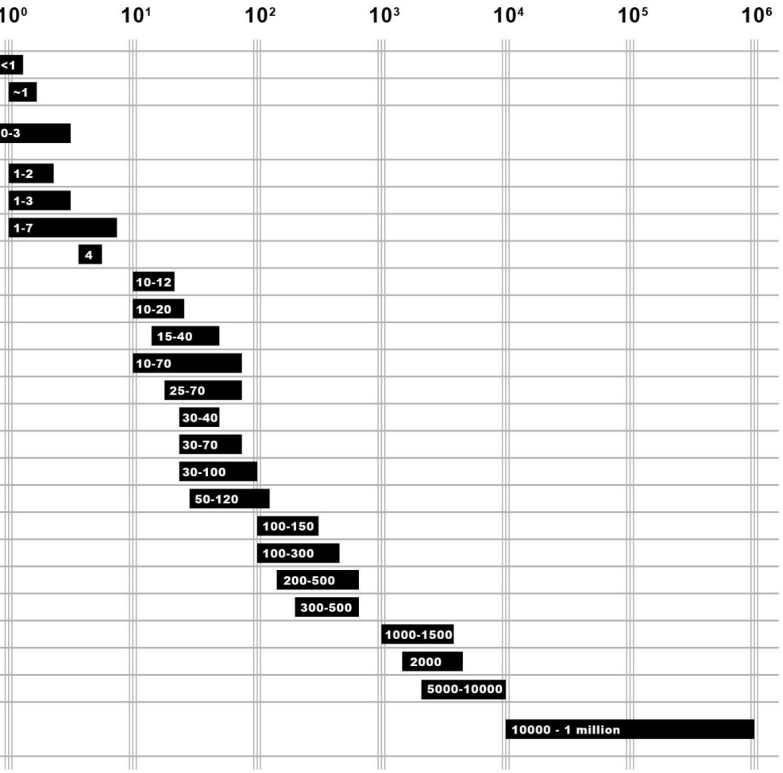
Example: `jmp *%eax`

© 2008–2018 by the MIT 6.172 Lecturers

Not all CPU operations are created equal

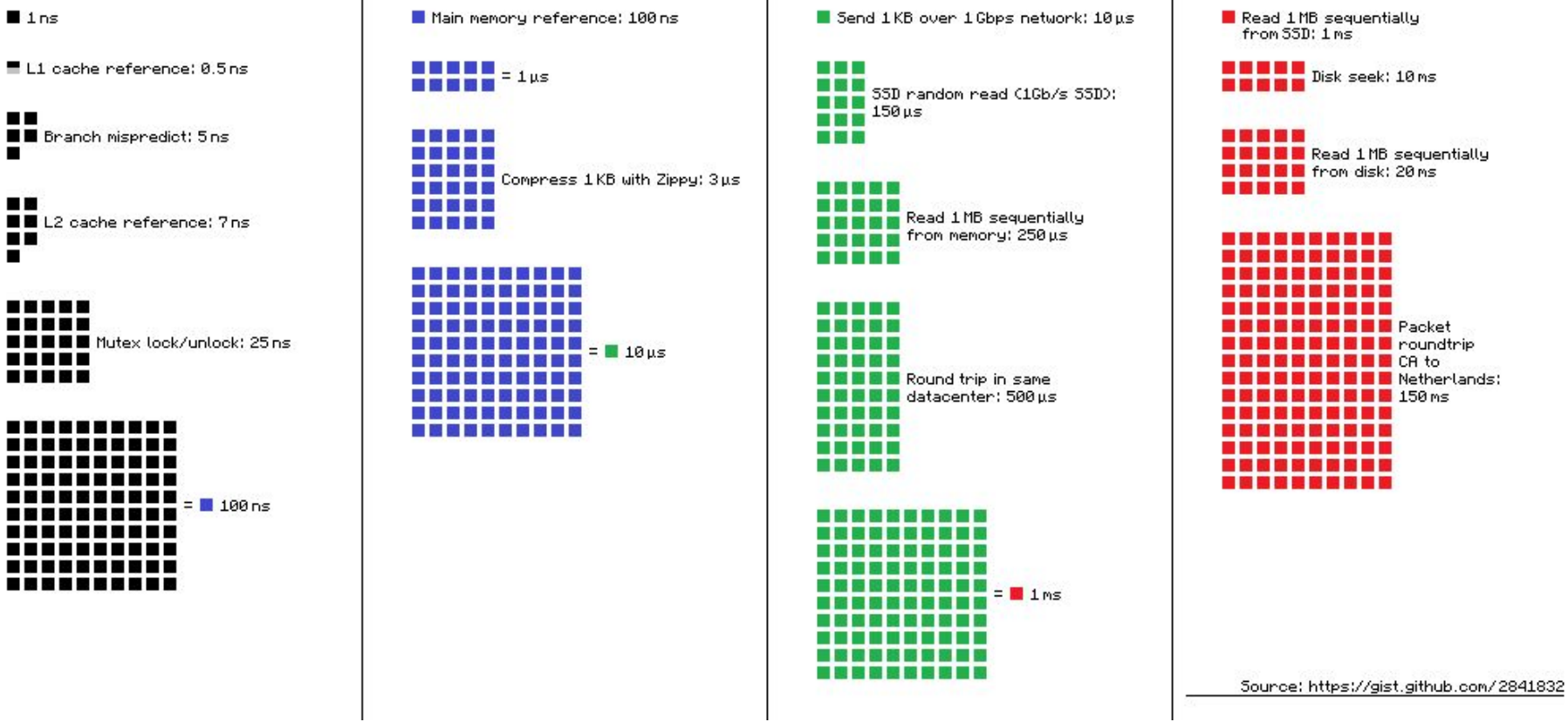
ithare.com

Operation	Cost in CPU Cycles
“Simple” register-register op (ADD,OR,etc.)	<1
Memory write	~1
Bypass delay: switch between integer and floating-point units	0-3
“Right” branch of “if”	1-2
Floating-point/vector addition	1-3
Multiplication (integer/float/vector)	1-7
L1 read	4
L2 read	10-12
“Wrong” branch of “if” (branch misprediction)	10-20
Integer division	15-40
128-bit vector division	10-70
C function direct call	25-70
Floating-point division	30-40
L3 read	30-70
C function indirect call	30-100
C++ virtual function call	50-120
Main RAM read	100-150
NUMA: different-socket L3 read	100-300
Allocation+deallocation pair (small objects)	200-500
NUMA: different-socket main RAM read	300-500
Kernel call	1000-1500
Thread context switch (direct costs)	2000
C++ Exception thrown+caught	5000-10000
Thread context switch (total costs, including cache invalidation)	10000 - 1 million



Distance which light travels while the operation is performed

Latency Numbers Every Programmer Should Know



Source: https://gist.github.com/2841832

Intel Assembler 80186 and higher

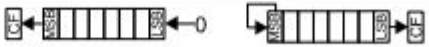
CodeTable 1/2

© 1996-2003 by Roger Jegerlehner, Switzerland
V 2.3 English. Also available in Spanish

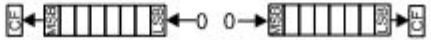
TRANSFER				Flags								
Name	**Comment**	**Code**	**Operation**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
MOV	Move (copy)	MOV Dest,Source	Dest:=Source									
XCHG	Exchange	XCHG Op1,Op2	Op1:=Op2 , Op2:=Op1									
STC	Set Carry	STC	CF:=1									1
CLC	Clear Carry	CLC	CF:=0									0
CMC	Complement Carry	CMC	CF:= ¬CF									±
STD	Set Direction	STD	DF:=1 (string op's downwards)		1							
CLD	Clear Direction	CLD	DF:=0 (string op's upwards)		0							
STI	Set Interrupt	STI	IF:=1			1						
CLI	Clear Interrupt	CLI	IF:=0			0						
PUSH	Push onto stack	PUSH Source	DEC SP, [SP]:=Source									
PUSHF	Push flags	PUSHF	O, D, I, T, S, Z, A, P, C 286+: also NT, IOPL									
PUSHA	Push all general registers	PUSHA	AX, CX, DX, BX, SP, BP, SI, DI									
POP	Pop from stack	POP Dest	Dest:=[SP], INC SP									
POPF	Pop flags	POPF	O, D, I, T, S, Z, A, P, C 286+: also NT, IOPL	±	±	±	±	±	±	±	±	±
POPA	Pop all general registers	POPA	DI, SI, BP, SP, BX, DX, CX, AX									
CBW	Convert byte to word	CBW	AX:=AL (signed)									
CWD	Convert word to double	CWD	DX:AX:=AX (signed)	±				±	±	±	±	±
CWDE	Conv word extended double	CWDE 386	EAX:=AX (signed)									
IN *i*	Input	IN Dest, Port	AL/AX/EAX := byte/word/double of specified port									
OUT *i*	Output	OUT Port, Source	Byte/word/double of specified port := AL/AX/EAX									

i for more information see instruction specifications

Flags: ±=affected by this instruction ?=undefined after this instruction

ARITHMETIC				Flags								
Name	**Comment**	**Code**	**Operation**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
ADD	Add	ADD Dest,Source	Dest:=Dest+Source	±				±	±	±	±	±
ADC	Add with Carry	ADC Dest,Source	Dest:=Dest+Source+CF	±				±	±	±	±	±
SUB	Subtract	SUB Dest,Source	Dest:=Dest-Source	±				±	±	±	±	±
SBB	Subtract with borrow	SBB Dest,Source	Dest:=Dest-(Source+CF)	±				±	±	±	±	±
DIV	Divide (unsigned)	DIV Op	Op=byte: AL:=AX / Op AH:=Rest	?				?	?	?	?	?
DIV	Divide (unsigned)	DIV Op	Op=word: AX:=DX:AX / Op DX:=Rest	?				?	?	?	?	?
DIV 386	Divide (unsigned)	DIV Op	Op=doublew.: EAX:=EDX:EAX / Op EDX:=Rest	?				?	?	?	?	?
IDIV	Signed Integer Divide	IDIV Op	Op=byte: AL:=AX / Op AH:=Rest	?				?	?	?	?	?
IDIV	Signed Integer Divide	IDIV Op	Op=word: AX:=DX:AX / Op DX:=Rest	?				?	?	?	?	?
IDIV 386	Signed Integer Divide	IDIV Op	Op=doublew.: EAX:=EDX:EAX / Op EDX:=Rest	?				?	?	?	?	?
MUL	Multiply (unsigned)	MUL Op	Op=byte: AX:=AL*Op if AH=0 ♦	±				?	?	?	?	±
MUL	Multiply (unsigned)	MUL Op	Op=word: DX:AX:=AX*Op if DX=0 ♦	±				?	?	?	?	±
MUL 386	Multiply (unsigned)	MUL Op	Op=double: EDX:EAX:=EAX*Op if EDX=0 ♦	±				?	?	?	?	±
IMUL *i*	Signed Integer Multiply	IMUL Op	Op=byte: AX:=AL*Op if AL sufficient ♦	±				?	?	?	?	±
IMUL	Signed Integer Multiply	IMUL Op	Op=word: DX:AX:=AX*Op if AX sufficient ♦	±				?	?	?	?	±
IMUL 386	Signed Integer Multiply	IMUL Op	Op=double: EDX:EAX:=EAX*Op if EAX sufficient ♦	±				?	?	?	?	±
INC	Increment	INC Op	Op:=Op+1 (Carry not affected !)	±				±	±	±	±	
DEC	Decrement	DEC Op	Op:=Op-1 (Carry not affected !)	±				±	±	±	±	
CMP	Compare	CMP Op1,Op2	Op1-Op2	±				±	±	±	±	±
SAL	Shift arithmetic left (= SHL)	SAL Op,Quantity		*i*				±	±	?	±	±
SAR	Shift arithmetic right	SAR Op,Quantity		*i*				±	±	?	±	±
RCL	Rotate left through Carry	RCL Op,Quantity		*i*								±
RCR	Rotate right through Carry	RCR Op,Quantity		*i*								±
ROL	Rotate left	ROL Op,Quantity		*i*								±
ROR	Rotate right	ROR Op,Quantity		*i*								±

i for more information see instruction specifications ♦ then CF:=0, OF:=0 else CF:=1, OF:=1

LOGIC				Flags								
Name	**Comment**	**Code**	**Operation**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
NEG	Negate (two-complement)	NEG Op	Op:=0-Op if Op=0 then CF:=0 else CF:=1	±				±	±	±	±	±
NOT	Invert each bit	NOT Op	Op:=¬Op (invert each bit)									
AND	Logical and	AND Dest,Source	Dest:=Dest∧Source	0				±	±	?	±	0
OR	Logical or	OR Dest,Source	Dest:=Dest∨Source	0				±	±	?	±	0
XOR	Logical exclusive or	XOR Dest,Source	Dest:=Dest (exor) Source	0				±	±	?	±	0
SHL	Shift logical left (= SAL)	SHL Op,Quantity		*i*				±	±	?	±	±
SHR	Shift logical right	SHR Op,Quantity		*i*				±	±	?	±	±

MISC				Flags								
Name	**Comment**	**Code**	**Operation**	**O**	**D**	**I**	**T**	**S**	**Z**	**A**	**P**	**C**
NOP	No operation	NOP	No operation									
LEA	Load effective address	LEA Dest,Source	Dest := address of Source									
INT	Interrupt	INT Nr	interrupts current program, runs spec. int-program			0	0					

JUMPS (flags remain unchanged)							
Name	**Comment**	**Code**	**Operation**	**Name**	**Comment**	**Code**	**Operation**
CALL	Call subroutine	CALL Proc		RET	Return from subroutine	RET	
JMP	Jump	JMP Dest					
JE	Jump if Equal	JE Dest	(= JZ)	JNE	Jump if not Equal	JNE Dest	(= JNZ)
JZ	Jump if Zero	JZ Dest	(= JE)	JNZ	Jump if not Zero	JNZ Dest	(= JNE)
JCXZ	Jump if CX Zero	JCXZ Dest		JECXZ	Jump if ECX Zero	JECXZ Dest	386
JP	Jump if Parity (Parity Even)	JP Dest	(= JPE)	JNP	Jump if no Parity (Parity Odd)	JNP Dest	(= JPO)
JPE	Jump if Parity Even	JPE Dest	(= JP)	JPO	Jump if Parity Odd	JPO Dest	(= JNP)

JUMPS Unsigned (Cardinal)				JUMPS Signed (Integer)			
JA	Jump if Above	JA Dest	(= JNBE)	JG	Jump if Greater	JG Dest	(= JNLE)
JAE	Jump if Above or Equal	JAE Dest	(= JNB = JNC)	JGE	Jump if Greater or Equal	JGE Dest	(= JNL)
JB	Jump if Below	JB Dest	(= JNAE = JC)	JL	Jump if Less	JL Dest	(= JNGE)
JBE	Jump if Below or Equal	JBE Dest	(= JNA)	JLE	Jump if Less or Equal	JLE Dest	(= JNG)
JNA	Jump if not Above	JNA Dest	(= JBE)	JNG	Jump if not Greater	JNG Dest	(= JLE)
JNAE	Jump if not Above or Equal	JNAE Dest	(= JB = JC)	JNGE	Jump if not Greater or Equal	JNGE Dest	(= JL)
JNB	Jump if not Below	JNB Dest	(= JAE = JNC)	JNL	Jump if not Less	JNL Dest	(= JGE)
JNBE	Jump if not Below or Equal	JNBE Dest	(= JA)	JNLE	Jump if not Less or Equal	JNLE Dest	(= JG)
JC	Jump if Carry	JC Dest		JO	Jump if Overflow	JO Dest	
JNC	Jump if no Carry	JNC Dest		JNO	Jump if no Overflow	JNO Dest	
				JS	Jump if Sign (= negative)	JS Dest	
				JNS	Jump if no Sign (= positive)	JNS Dest	